

Ejercicio 3 Sistema de Control de Órdenes de Servicio de un Taller Automotriz

#	Tipo de dato	Nombre	Visibilidad	Motivo	Clase
1	int	numero Orden	private	Almacenar el número identificador de la orden de servicio.	OrdenServicio
2	String	propietario	private	Almacenar el nombre del dueño del vehículo asociado a la orden.	OrdenServicio
3	String	placa	private	Almacenar la placa para identificar el vehículo a reparar.	OrdenServicio
4	String	descripcion	private	Guardar el detalle del trabajo o servicio que se realizará.	OrdenServicio
5	double	costoEstimado	private	Guardar el valor monetario proyectado para el servicio.	OrdenServicio
6	List<OrdenServicio>	ordenes	private	Colección dinámica que administra las órdenes registradas sin un límite fijo.	Taller
7	Scanner	scanner	private	Leer y capturar todos los datos ingresados por el usuario en la consola.	Vista
8	Vista	vista	private	Permitir al controlador solicitar datos al usuario e imprimir resultados.	Controlador
9	Taller	taller	private	Permitir al controlador acceder a la lógica de negocio y modificar la colección de órdenes.	Controlador

#	Retorno	Nombre	Parámetros	Visibilidad	Motivo	Clase
1	OrdenServicio	OrdenServicio	int numero, String prop, String placa, String desc, double costo	public	Constructor: Inicializar la orden con los datos validados.	OrdenServicio
2	int	getNumeroOrden	Ninguno	public	Obtener el número de orden para validaciones y búsquedas.	OrdenServicio
3	String	getPlaca	Ninguno	public	Obtener la placa para agrupar las coincidencias.	OrdenServicio
4	double	getCostoEstimado	Ninguno	public	Obtener el costo estimado para	OrdenServicio

					realizar los cálculos financieros.	
5	void	setDescripcion	String descripcion	public	Permitir la modificación de la descripción del servicio.	OrdenServicio
6	void	setCostoEstimado	double costo	public	Permitir la modificación del costo estimado de la orden.	OrdenServicio
7	String	toString	Ninguno	public	Devolver la representación en texto de todos los datos de la orden.	OrdenServicio
8	void	agregarOrden	OrdenServicio orden	public	Añadir una orden a la colección dinámica List.	Taller
9	boolean	existeOrden	int numero	public	Verificar si un número de orden ya está registrado para evitar duplicados.	Taller
10	List<OrdenServicio>	obtenerTodas	Ninguno	public	Devolver la lista completa de órdenes para su consulta.	Taller
11	OrdenServicio	buscarPorNumero	int numero	public	Encontrar una orden por su número o lanzar una excepción si no existe.	Taller
12	void	modificarOrden	int numero, String desc, double costo	public	Actualizar descripción y costo de una orden previamente registrada.	Taller
13	void	cancelarOrden	int numero	public	Eliminar una orden	Taller

					específica de la colección dinámica.	
14	List<OrdenServicio>	buscarPorPlaca	String placa	public	Devolver todas las órdenes que coincidan con la placa solicitada.	Taller
15	double	calcularTotal	Ninguno	public	Sumar los costos estimados de todas las órdenes activas.	Taller
16	double	calcularPromedio	Ninguno	public	Dividir el costo total entre la cantidad actual de órdenes.	Taller
17	OrdenServicio	obtenerMayorCosto	Ninguno	public	Recorrer la colección para determinar cuál orden es la más costosa.	Taller
18	int	obtenerCantidad	Ninguno	public	Indicar cuántas órdenes se encuentran almacenadas actualmente.	Taller
19	int	mostrarMenu	Ninguno	public	Mostrar las 10 opciones requeridas y leer la elección.	Vista
20	String	pedirTexto	String mensaje	public	Imprimir un mensaje y capturar un texto ingresado por el usuario.	Vista
21	double	pedirDouble	String mensaje	public	Capturar un valor decimal para procesar los costos estimados.	Vista

2					Capturar un número entero para el número de orden o menú.	
2	int	pedirEntero	String mensaje	public		Vista
2					Imprimir resultados de búsquedas, estadísticas o errores en pantalla.	
3	void	mostrarMensaje	String mensaje	public		Vista
2					Ejecutar el ciclo principal, manejar el try-catch general y el bloque finally.	
4	void	iniciar	Ninguno	public		Controlador

Preguntas

1) ¿Qué clases son necesarias para representar el problema planteado?

Se necesitan las clases OrdenServicio para representar los datos, Taller para administrar la colección dinámica y la lógica de negocio, Vista para la interfaz de usuario, Controlador para manejar el flujo y las excepciones y Main para iniciar la ejecución del programa.

4) ¿Qué información deberá almacenarse dentro de la colección dinámica?

Se almacenarán las direcciones de memoria que apuntan a los objetos instanciados de la clase OrdenServicio. Con estas referencias, la estructura permite acceder y gestionar de forma dinámica toda la información encapsulada en los objetos.

5) ¿Dónde utilizará List y dónde realizará la creación mediante ArrayList?

Se utilizará la interfaz List para declarar el tipo del atributo de la colección dentro de la clase Taller. La creación se realizará instanciando la clase ArrayList dentro del constructor de esa misma clase Taller.

8) ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores iniciales les asignará?

Los objetos OrdenServicio recibirán sus valores iniciales a través de su constructor. Los valores asignados serán los datos específicos que el usuario ingrese y que sean validados por el sistema.

9) ¿Cómo identificará una orden específica dentro de la colección?

Se identificará mediante un recorrido de la colección dinámica, comparando el número de orden que se desea buscar con el atributo de número de orden de cada objeto almacenado hasta encontrar una coincidencia.

10) ¿Cómo agregará y eliminará de la colección dinámica?

Las órdenes se agregarán utilizando el método `.add()` de la lista dinámica, y se eliminarán utilizando el método `.remove()`, proporcionando el objeto correspondiente que se encontró previamente en la búsqueda.

11) ¿Cómo realizará la búsqueda de todas las órdenes asociadas a una misma placa?

Se iterará sobre toda la lista comparando la placa solicitada con la placa registrada en cada orden. Las coincidencias encontradas se agruparán en una lista secundaria o se imprimirán directamente para mostrar todas las órdenes asociadas a ese vehículo.

12) ¿Cómo recorrerá la colección para calcular el valor total y el costo promedio de las órdenes?

Se utilizará un ciclo para recorrer la colección, sumando el costo estimado de cada orden activa en una variable acumuladora para obtener el valor total. Luego, el costo promedio se calculará dividiendo este valor total entre la cantidad actual de órdenes utilizando el método `.size()` de la lista.

13) ¿Cómo determinará cuál es la orden con el costo estimado más alto?

Se recorrerá la colección comparando el costo de cada orden con una variable temporal que almacenará el valor máximo encontrado. Si se encuentra un costo superior, se actualizará esta variable y se guardará la referencia a la orden correspondiente.

14) ¿Qué situaciones dentro del programa pueden producir una excepción?

Las excepciones se pueden producir al ingresar datos con un formato incorrecto o al intentar buscar, modificar o cancelar una orden de servicio que no existe en el sistema (saliéndose del límite del `ArrayList` o buscando un valor `null`).

15) ¿Qué operaciones serán protegidas utilizando try-catch?

Se protegerán las lecturas de datos por consola que requieran formatos numéricos, y los procesos de búsqueda, modificación y cancelación de órdenes.

16) ¿Qué acción realizará mediante el bloque finally y por qué debe ejecutarse independientemente del resultado de la operación?

El bloque finally se utilizará para ejecutar una instrucción de limpieza o para mostrar un mensaje que indique que la operación concluyó y que el programa retornará al menú principal. Debe ejecutarse siempre para asegurar que, sin importar si ocurrió un error o si el proceso fue exitoso, el sistema mantenga su flujo y estabilidad. De esta forma asegurando que, en caso de una excepción, el programa no muera.